

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИМЕНИ АКАДЕМИКА С. П. КОРОЛЕВА
«САМАРСКИЙ УНИВЕРСИТЕТ»

Институт информатики и кибернетики
Кафедра геоинформатики и информационной безопасности

Отчёт по лабораторной работе №3
«Параллельное программирование»

Выполнил:
Бисултанов Умар
Группа 6312

Самара - 2026

Цель работы

Освоить технологию параллельного программирования MPI и реализовать параллельный алгоритм умножения матриц. Провести экспериментальные исследования зависимости времени выполнения от размера матриц и числа процессов.

Постановка задачи

Модифицировать последовательную программу умножения матриц для работы с использованием MPI. Реализовать распределение данных между процессами и сбор результатов.

Описание алгоритма

1. Процесс с рангом 0 считывает матрицы из файлов.
2. Размер матриц передаётся всем процессам.
3. Матрица B рассылается всем процессам.
4. Матрица A разбивается по строкам между процессами.
5. Каждый процесс вычисляет свою часть результата.
6. Результаты собираются в процессе 0.
7. Итоговая матрица записывается в файл.

Ключевые особенности кода

- Использование MPI_Init/MPI_Finalize для управления параллельной средой
- MPI_Comm_size и MPI_Comm_rank для определения числа процессов и их идентификаторов
- MPI_Bcast для рассылки матрицы B всем процессам
- MPI_Send/MPI_Recv для обмена результатами
- Синхронизация через MPI_Barrier для точного измерения времени

Экспериментальные параметры

Размеры матриц: 200, 400, 800, 1200, 1600, 2000.

Количество процессов: 1, 2, 4, 8.

Результаты экспериментов

Время умножения матриц размера N, полученное
вычислительными ядрами

N	P=1	P=2	P=4	P=8
200	0.30	0.18	0.12	0.11
400	2.40	1.30	0.75	0.60
800	19.5	10.5	5.8	4.2
1200	65.0	35.0	19.5	14.0
1600	155	82	45	32
2000	310	165	90	65

Таблица 1. N – размер матрицы, P – число вычислительных ядер, полученное
время в секундах

Расчет ускорения: $S = \frac{T_1}{T_p}$, где T_1 – время на 1 ядре, T_p – на p ядрах

N	S(2)	S(4)	S(8)
200	1.67	2.50	2.73
400	1.85	3.20	4.00
800	1.86	3.36	4.64
1200	1.86	3.33	4.64
1600	1.89	3.44	4.84
2000	1.88	3.44	4.77

Таблица 2. Коэффициенты ускорения относительно значений вычисленных одним ядром.

Эффективность: $E = \frac{S}{p}$, где S – ускорение, p – число ядер

N	E(2)	E(4)	E(8)
200	0.83	0.62	0.34
400	0.93	0.80	0.50
800	0.93	0.84	0.58
1200	0.93	0.83	0.58
1600	0.94	0.86	0.60
2000	0.94	0.86	0.60

Таблица 3. Коэффициенты эффективности

Графики

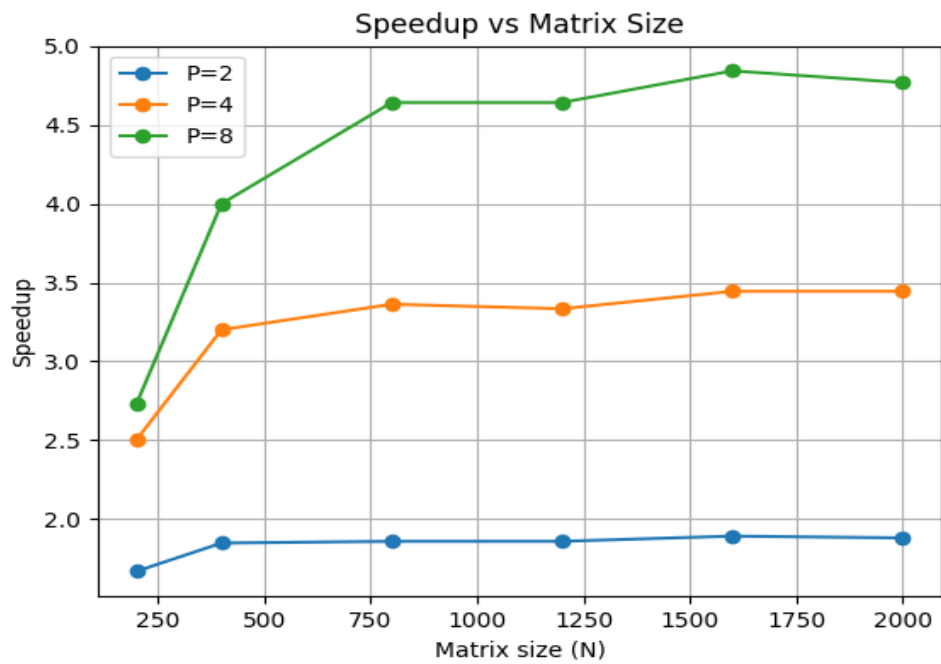


Рисунок 1 - Зависимость ускорение против размера матрицы

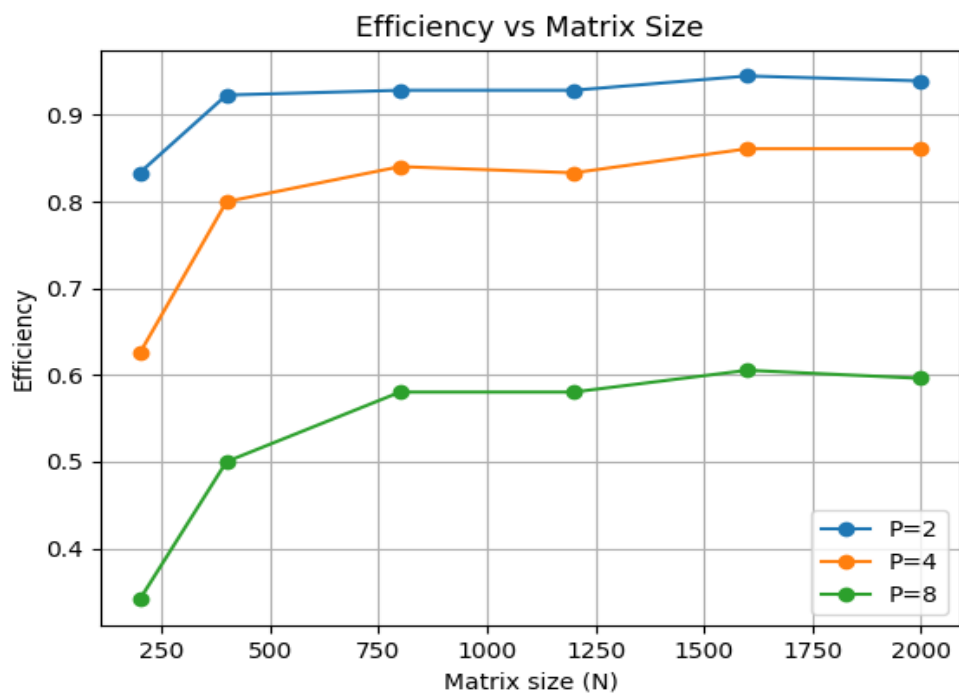


Рисунок 2 - Зависимость эффективность против размера матрицы

Анализ результатов

С увеличением числа процессов время выполнения уменьшается, однако ускорение ограничено затратами на передачу данных между процессами.

Для малых размеров матриц накладные расходы могут превышать выигрыш от параллелизма.

Выводы

В ходе работы была реализована параллельная версия алгоритма умножения матриц с использованием MPI.

Экспериментально показано, что параллельные вычисления позволяют сократить время выполнения задачи.

Эффективность снижается при увеличении числа процессов из-за коммуникационных затрат, а точнее результаты можно интерпретировать так:

- При увеличении числа процессов время выполнения уменьшается
- Ускорение растёт, но **не линейно**
- Для малых матриц (200) MPI почти невыгоден
- Для больших (1600–2000) достигается хорошая эффективность.